Java SAST Engine — 사용자 가이드

- Java SAST Engine — 사용자 가이드
  - 목차
  - 1. 도구 소개
    - 준수 기준
    - 탐지 가능한 주요 보안약점
    - 기술 스택
  - 2. 설치 및 환경 설정
    - 2.1 사전 요구사항
    - 2.2 JDK 17 설치 (Amazon Linux 2023 / EC2 기준)
    - 2.3 Maven 설치
    - 2.4 소스 빌드
    - 2.5 EC2에서 웹 서버로 실행
  - 3. 주요 기능
    - 3.1 69개 항목별 상세 가이드 (Bad/Good Code 비교)
    - 3.2 지능형 Taint Analysis (Sanitizer 인식 포함)
    - 3.3 중복 탐지 방지
    - 3.4 HTML 대시보드 (웹 UI)
    - 3.5 PDF 리포트 생성
  - 4. 사용 방법
    - 4.1 웹 UI 모드 (권장)
    - 4.2 CLI 모드 (서버리스 / 자동화)
    - 4.3 Maven 명령어 정리
    - 4.4 분석 대상 압축 방법
  - 5. 결과 확인
    - 5.1 웹 대시보드 (`results.html`) 구성
    - 5.2 상세 모달 (Bad/Good Code 비교)
    - 5.3 위험도 판정 기준
    - 5.4 오염 흐름(Taint Flow) 해석

- 5.5 CLI 리포트 (Markdown) 예시
- 6. 규칙 관리
  - 6.1 규칙 파일 위치
  - 6.2 Rule ID 체계
  - 6.3 규칙 구조 예시
  - 6.4 규칙 필드 설명
  - 6.5 새 규칙 추가 절차
  - 6.6 severity 값 변경 예시
- 7. 오탐 억제
  - 7.1 sast-suppressions.json (전역 억제)
  - 7.2 @SuppressWarnings 어노테이션 (코드 수준 억제)
  - 7.3 테스트 코드 자동 처리
- 8. 자주 묻는 질문

Java SAST Engine — 사용자 가이드

> **행정안전부 소프트웨어 보안약점 진단가이드 (2021) 기반**
> PART3 설계단계 20개 · PART4 구현단계 49개 — 총 **69개 보안 규칙** 자동 탐지

목차

1. 도구 소개
2. 설치 및 환경 설정
3. 주요 기능
4. 사용 방법
5. 결과 확인
6. 규칙 관리
7. 오탐 억제
8. 자주 묻는 질문

---

1. 도구 소개

Java SAST Engine은 Java 소스코드를 정적 분석(SAST, Static Application Security Testing)하여 보안 약점을 자동으로 탐지·보고하는 도구입니다.

준수 기준

파트	분류	규칙 수	Rule ID 형식
PART3 — 분석·설계단계	설계 보안 기준	20개	`DS-x.x`
PART4 — 구현단계	보안약점 제거 기준	49개	`IV-x.x`

탐지 가능한 주요 보안약점

위험도	대표 약점	Rule ID
CRITICAL	OS 명령어 삽입, 역직렬화 취약점	IV-1.5, IV-5.5
HIGH	SQL 삽입, 경로 조작, XSS, LDAP 삽입	IV-1.1, IV-1.3, IV-1.4, IV-1.10
MEDIUM	HTTP 응답분할, SSRF, 오류 메시지 노출	IV-1.12, IV-1.13, IV-4.1
LOW	디버그 코드, 주석 내 중요 정보	IV-6.2, IV-2.13

기술 스택

- **Runtime:** Java 17, Spring Boot 3.3
- **AST 파서:** JavaParser 3.25.x + SymbolSolver
- **리포트:** Markdown, JSON, PDF (OpenPDF — 한국어 지원)
- **웹 UI:** Spring MVC + Thymeleaf + Bootstrap 5

2. 설치 및 환경 설정

2.1 사전 요구사항

항목	최소 버전	확인 명령
JDK	17 이상	`java -version`
Maven	3.8 이상	`mvn -version`
메모리	512MB 이상 권장	—

참고: JDK 21을 사용해도 동작합니다. Spring Boot 3.x는 JDK 17+를 요구합니다.

2.2 JDK 17 설치 (Amazon Linux 2023 / EC2 기준)

```
# Corretto 17 설치
sudo dnf install -y java-17-amazon-corretto-devel

# 설치 확인
java -version
# openjdk version "17.x.x" ...
```

2.3 Maven 설치

```
# Maven 설치
sudo dnf install -y maven

# 설치 확인
mvn -version
# Apache Maven 3.x.x
```

2.4 소스 빌드

```
# 프로젝트 디렉터리로 이동
cd java-sast-tool

# 전체 빌드 (테스트 포함)
mvn clean package

# 빠른 빌드 (테스트 생략)
mvn clean package -DskipTests

# 빌드 결과물 확인
ls target/sast.jar
```

빌드 성공 시 `target/sast.jar` (Fat JAR)가 생성됩니다.

2.5 EC2에서 웹 서버로 실행

```
# 포트 5000 허용 (보안 그룹에서 TCP 5000 인바운드 추가 필요)
# 웹 서버 시작 (포트 5000, 백그라운드 실행)
nohup java -jar target/sast.jar > sast.log 2>&1 &

# 시작 확인
tail -f sast.log
# ... Started SastApplication in X.XXX seconds
```

브라우저에서 `http://15.164.190.193:5000` 에 접속합니다.

> **주의:** EC2 보안 그룹에서 TCP 포트 5000 인바운드 규칙을 허용해야 합니다.

---

3. 주요 기능

3.1 69개 항목별 상세 가이드 (Bad/Good Code 비교)

`security-rules.json`에 정의된 모든 69개 규칙은 다음 정보를 포함합니다.

- **탐지 설명:** 왜 취약한지 한국어로 기술
- **Bad Code 예시:** 취약한 패턴의 실제 Java 코드
- **Good Code 예시:** 안전하게 수정된 Java 코드
- **가이드 참조:** 행정안전부 가이드 페이지 번호

- **CWE 번호:** 국제 표준 취약점 분류 번호
- **참고 URL:** CWE 미트레 링크

결과 대시보드에서 각 탐지 항목의 **"상세"** 버튼을 클릭하면 해당 규칙의 Bad/Good Code를 탭 형식으로 나란히 확인할 수 있습니다.

3.2 지능형 Taint Analysis (Sanitizer 인식 포함)

AST(추상 구문 트리) 기반 **인트라-프로시저럴 Dataflow 분석**으로 오염값의 흐름을 추적합니다.

```
[Source] 외부 입력 진입
    ↓  request.getParameter(), getenv(), readLine() 등
[Propagator] 변수 간 오염 전파
    ↓  String sql = "SELECT..." + userInput
[Sanitizer] 오염 무력화 인식 → 탐지 제외
    ↓  prepareStatement(), escapeHtml(), getCanonicalPath() 등
[Sink] 위험 API 도달 시 취약점 보고
    ↓  executeQuery(), exec(), print() 등
```

인식 가능한 Sanitizer 목록:

분류	메서드
SQL 파라미터화	`prepareStatement`, `setString`, `setInt`, `setObject` 등
HTML/XSS 인코딩	`escapeHtml`, `encodeForHTML`, `sanitizeHtml`, `htmlEscape` 등
경로 정규화	`getCanonicalPath`, `normalize`, `toRealPath`
LDAP/XPath	`encodeForLDAP`, `encodeForXPath`
범용 검증	`validate`, `sanitize`, `canonicalize`, `validateInput`

Sanitizer가 호출된 경우 해당 변수는 오염 상태에서 제거되어 **오탐(False Positive)을 방지**합니다.

3.3 중복 탐지 방지

동일 파일의 동일 라인·동일 규칙에 대한 중복 탐지 결과를 자동으로 제거합니다. Taint 분석(AST 기반)과 패턴 분석(정규식 기반)이 같은 코드를 동시에 탐지해도 한 건으로만 보고합니다.

3.4 HTML 대시보드 (웹 UI)

ZIP 또는 7z 파일 업로드만으로 전체 프로젝트를 분석하고 결과를 웹 브라우저에서 확인합니다.

- 위험도별 요약 카드 (CRITICAL / HIGH / MEDIUM / LOW)
- 탐지 결과 아코디언 리스트 (규칙 ID, 파일명, 라인, 코드 스니펫)
- 오염 흐름(Taint Flow) 시각화: Source → Propagator → Sink
- 항목별 Bad/Good Code 탭 비교
- PDF 다운로드 버튼 (한국어 지원)

3.5 PDF 리포트 생성

분석 완료 후 웹 대시보드의 **"PDF 다운로드"** 버튼 또는 CLI 실행 시 자동으로 PDF 파일이 생성됩니다. PDF에는 다음이 포함됩니다.

- 분석 요약 (총 탐지 건수, 위험도별 분류)
- 각 탐지 항목의 취약 코드 및 수정 권고 코드
- 가이드 참조 페이지 및 CWE 번호
- 한국어 폰트 지원 (나눔고딕 임베딩)

---

4. 사용 방법

4.1 웹 UI 모드 (권장)

1. 브라우저에서 `http://15.164.190.193:5000` 접속
2. 분석할 Java 프로젝트를 **ZIP 또는 7z**으로 압축
3. 업로드 영역에 파일을 드래그하거나 클릭하여 선택
4. **"분석 시작"** 버튼 클릭
5. 분석 완료 후 결과 대시보드(`results.html`)로 자동 이동

> 최대 업로드 파일 크기: **100MB**

4.2 CLI 모드 (서버리스 / 자동화)

웹 서버 없이 명령줄에서 직접 실행합니다.

```
# 기본 실행: <소스 디렉터리> <리포트 출력 경로>
java -cp target/sast.jar com.sast.SASTEngine <소스_디렉터리> <리포트_파일.md>

# 예시: 현재 프로젝트 분석
java -cp target/sast.jar com.sast.SASTEngine ./src/main/java ./report.md

# 예시: 특정 프로젝트 분석
java -cp target/sast.jar com.sast.SASTEngine /home/ec2-user/myproject/src ./output/
        report.md
```

CLI 실행 시 다음 파일이 자동 생성됩니다.

파일	내용
`report.md`	Markdown 형식 상세 리포트
`report.json`	JSON 형식 (CI/CD 연동용)
`report.pdf`	PDF 형식 (한국어)

4.3 Maven 명령어 정리

```
# 전체 빌드 + 테스트
mvn clean package

# 테스트만 실행
mvn test

# 특정 테스트 클래스만 실행
mvn test -Dtest=SanitizerFalsePositiveTest

# 코드 커버리지 리포트 생성
mvn test jacoco:report
open target/site/jacoco/index.html

# Fat JAR 위치
target/sast.jar
```

4.4 분석 대상 압축 방법

```
# Maven 프로젝트 압축 예시 (src 디렉터리만)
cd /path/to/your-project
zip -r project-src.zip src/

# 프로젝트 전체 압축 (node_modules, .git 제외)
zip -r project.zip . --exclude "*.git*" --exclude "*/node_modules/*" --exclude "*/
        target/*"

# 7z 형식으로 압축
7z a project.7z src/
```

---

5. 결과 확인

5.1 웹 대시보드 (`results.html`) 구성

분석 완료 후 표시되는 대시보드는 다음과 같이 구성됩니다.

```
┌──────────────────────────────────────────────────────────┐
│  Java SAST Engine              [PDF 다운로드] [새 분석]      │
├──────────────────────────────────────────────────────────┤
│  project.zip — 분석 파일 수: 12개 · 총 탐지: 8건            │
│                                                            │
│  [  CRITICAL: 1]  [  HIGH: 3]  [  MEDIUM: 2]  [○ LOW: 2] │
├──────────────────────────────────────────────────────────┤
│  ▼ [HIGH] IV-1.1 — SQL 삽입     BoardController.java│
│    Line 42 · CWE-89                          [상세 보기 →]   │
│    ┌──────────────────────────────────────────────┐  │
│    │ String sql = "SELECT * FROM user WHERE id='" + id;  │  │
│    └──────────────────────────────────────────────┘  │
│    오염 흐름: [Source L3: getParameter("id")] → [sql] →      │
│              [Sink L42: executeQuery(sql)]                 │
├──────────────────────────────────────────────────────────┤
│  ▼ [HIGH] IV-1.3 — 경로 조작     FileController.java │
│  ...                                                       │
└──────────────────────────────────────────────────────────┘
```

5.2 상세 모달 (Bad/Good Code 비교)

각 탐지 항목에서 **"상세 보기"** 버튼 클릭 시 모달 창이 열립니다.

```
┌──────────────────────── 상세 정보 ────────────────────────┐
│  IV-1.1 — SQL 삽입                              [CWE-89]  │
│  PART4 제1절 1항 (p.180-193)                              │
├──────────────────────────────────────────────────────────┤
│  탐지 근거                                                │
│  외부 입력값 'id'(request.getParameter("id"), L3)이       │
│  검증 없이 stmt.executeQuery(sql)(L42)에 사용됩니다.      │
├──────────────────────────────────────────────────────────┤
│  권고 수정 가이드                                         │
│  [ 나쁜 예시]  [ 좋은 예시]                               │
│                                                           │
│  나쁜 예시                                                │
│  ┌──────────────────────────────────────────────────┐    │
│  │ String sql = "SELECT * FROM user WHERE id='" + id;  │
│  │ Statement stmt = con.createStatement();          │    │
│  │ ResultSet rs = stmt.executeQuery(sql);           │    │
│  └──────────────────────────────────────────────────┘    │
│                                                           │
│  좋은 예시                                                │
│  ┌──────────────────────────────────────────────────┐    │
│  │ String sql = "SELECT * FROM user WHERE id = ?";  │    │
│  │ PreparedStatement ps = con.prepareStatement(sql);  │    │
│  │ ps.setString(1, id);                             │    │
│  │ ResultSet rs = ps.executeQuery();                │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

5.3 위험도 판정 기준

위험도	아이콘	판정 기준
CRITICAL		원격 코드 실행, 시스템 완전 장악 가능
HIGH		인증 우회, 데이터 탈취, 경로 탈출 가능
MEDIUM		제한적 정보 노출, 서비스 장애 가능
LOW	○	잠재적 위험, 직접 악용 어려움

> 테스트 코드(`src/test/` 경로) 내 탐지 항목은 자동으로 **LOW**로 하향 조정됩니다.

5.4 오염 흐름(Taint Flow) 해석

```
[Source L3: request.getParameter("id")] → [Propagators: id, sql] → [Sink L42: stmt.executeQuery(sql)]
```

요소	의미
Source	외부 입력이 코드로 진입하는 지점 (라인 번호 포함)
Propagators	오염값이 거쳐 간 변수 이름 목록
Sink	오염값이 위험 API에 도달한 지점 (라인 번호 포함)

5.5 CLI 리포트 (Markdown) 예시

```
### [1] IV-1.1 — SQL 삽입

| 항목 | 내용 |
|------|------|
| **위험도** | 높음(HIGH) |
| **파일** | `com/example/BoardController.java` |
| **라인** | 42 |
| **CWE** | CWE-89 |

#### 오염 흐름
[Source L3: request.getParameter("id")] → [Propagators: [id, sql]] → [Sink L42:
        stmt.executeQuery(sql)]

#### 취약한 코드
...

#### 권고 수정 코드
...
```

---

6. 규칙 관리

6.1 규칙 파일 위치

```
src/main/resources/security-rules.json
```

이 파일에 행정안전부 가이드의 69개 규칙이 모두 정의되어 있습니다. 수정 후에는 **반드시 재빌드**(`mvn clean package`)가 필요합니다.

6.2 Rule ID 체계

```
DS-{절}.{항}  → PART3 설계단계  (예: DS-1.1, DS-2.3)
IV-{절}.{항}  → PART4 구현단계  (예: IV-1.1, IV-4.1)
```

6.3 규칙 구조 예시

```
{
  "ruleId": "IV-1.1",
  "part": "PART4",
  "section": "입력데이터 검증 및 표현",
  "name": "SQL 삽입",
  "cwe": ["CWE-89"],
  "severity": "HIGH",
  "phase": "IMPLEMENTATION",
  "taintAnalysis": true,
  "sources":    ["getParameter", "getenv", "readLine"],
  "sinks":      ["executeQuery", "executeUpdate", "execute"],
  "sanitizers": ["prepareStatement", "setString"],
  "dangerousPatterns": ["String sql = .* \\+ .*"],
  "remediation": {
    "strategy":   "USE_PREPARED_STATEMENT",
    "description": "SQL 질의문에 외부 입력값이 직접 포함되면 ...",
    "guideRef":   "PART4 제1절 1항 (p.180-193)",
    "bad_code":   "// 취약한 코드 예시 \nString sql = \"SELECT * FROM user WHERE id='\" +
        id + \"'\";",
    "good_code":  "// 안전한 코드 예시 \nPreparedStatement ps =
        con.prepareStatement(\"SELECT * FROM user WHERE id = ?\");\nps.setString(1,
        id);",
    "reference_url": "https://cwe.mitre.org/data/definitions/89.html"
  }
}
```

6.4 규칙 필드 설명

필드	필수	설명
`ruleId`		규칙 고유 ID (`DS-x.x` 또는 `IV-x.x`)
`name`		가이드 공식 항목명 (한국어)
`cwe`		CWE 번호 배열
`severity`		`CRITICAL` / `HIGH` / `MEDIUM` / `LOW`
`taintAnalysis`		`true` → AST Taint 분석, `false` → 정규식 패턴 탐지
`sources`	taint 규칙	Source API 메서드명 목록
`sinks`	taint 규칙	Sink API 메서드명 목록
`sanitizers`	taint 규칙	Sanitizer API 메서드명 목록
`dangerousPatterns`	pattern 규칙	Java 정규식 패턴 (소스 텍스트 스캔)
`remediation.bad_code`		취약 코드 예시 (대시보드에 표시)
`remediation.good_code`		안전 코드 예시 (대시보드에 표시)
`remediation.guideRef`		가이드 참조 (페이지 번호 포함)

6.5 새 규칙 추가 절차

```
1. security-rules.json 에 규칙 추가 (위 필드 모두 채울 것)
2. taintAnalysis: true 인 경우 → TaintAnalysisEngine.RULE_SINKS에 ruleId 매핑 추가
3. RemediationService 에 새 strategy 처리 로직 추가
4. mvn clean package 로 재빌드
5. 취약/안전 Java 픽스처 파일 작성 후 단위 테스트 실행
```

6.6 severity 값 변경 예시

특정 규칙의 위험도를 조직 정책에 맞게 조정합니다.

```
// 변경 전
{ "ruleId": "IV-6.2", "severity": "LOW" }

// 변경 후: 내부 정책상 디버그 코드를 MEDIUM으로 상향
{ "ruleId": "IV-6.2", "severity": "MEDIUM" }
```

변경 후 `mvn clean package` 로 재빌드하면 적용됩니다.

---

7. 오탐 억제

7.1 sast-suppressions.json (전역 억제)

`src/main/resources/sast-suppressions.json` 에 억제 규칙을 추가하면 특정 파일·규칙·라인의 탐지 결과를 제외합니다.

```
{
  "suppressions": [
    {
      "file": "LegacyBoardController.java",
      "ruleId": "IV-1.1",
      "line": null,
      "reason": "레거시 모듈 — 다음 스프린트 리팩터링 예정 (2024-Q4)"
    },
    {
      "file": "AdminController.java",
      "ruleId": null,
      "line": 88,
      "reason": "88번째 줄 전체 억제 — 내부 관리자 전용 API"
    }
  ]
}
```

필드	설명
`file`	파일명의 일부 문자열 (부분 일치)
`ruleId`	억제할 규칙 ID (`null` 이면 모든 규칙)
`line`	억제할 라인 번호 (`null` 이면 파일 전체)
`reason`	억제 사유 (로그에 기록됨)

억제 적용 후 **재빌드 없이** 서버를 재시작하면 반영됩니다.

7.2 @SuppressWarnings 어노테이션 (코드 수준 억제)

특정 메서드에 어노테이션을 추가하면 해당 메서드 내부의 탐지 결과를 무시합니다.

```
@SuppressWarnings("sast-ignore")
public void legacyMethod() {
    // 이 메서드 내부의 탐지 결과는 보고되지 않음
    String sql = "SELECT..." + param; // 억제됨
}
```

> **주의:** `sast-ignore` 억제는 실제 취약점을 숨기는 것이므로, 충분한 검토 후 사용하고 반드시 코드 리뷰에서 확인하세요.

7.3 테스트 코드 자동 처리

`src/test/` 경로 하위 파일에서 탐지된 항목은 자동으로 위험도가 **LOW**로 하향 조정됩니다. 테스트 코드는 의도적으로 취약 패턴을 포함하는 경우가 많기 때문입니다.

---

8. 자주 묻는 질문

Q. ZIP 업로드 후 "분석 중 오류가 발생했습니다"가 뜹니다.
A. 파일 크기가 100MB를 초과하거나, `.java` 파일이 없는 ZIP일 수 있습니다. ZIP 내부에 Java 소스 파일이 포함되어 있는지 확인하세요.

Q. 탐지 결과가 너무 많이 나옵니다 (오탐 의심).
A. `sast-suppressions.json`에 억제 규칙을 추가하거나, 코드에 `@SuppressWarnings("sast-ignore")`를 추가하세요. 또한 Sanitizer 메서드를 사용하고 있다면 `security-rules.json`의 `sanitizers` 배열에 해당 메서드명을 추가하면 오탐이 줄어듭니다.

Q. PDF가 생성되지 않거나 한국어가 깨집니다.
A. PDF 생성은 OpenPDF 라이브러리를 사용합니다. 서버에 한국어 폰트(나눔고딕)가 없는 경우 폴백 폰트로 대체될 수 있습니다. `sast.log`에서 PDF 관련 경고를 확인하세요.

Q. CI/CD 파이프라인에 통합하려면 어떻게 하나요?
A. CLI 모드(`java -cp target/sast.jar com.sast.SASTEngine`)를 사용하고, 생성된 `report.json`을 파싱하여 CRITICAL/HIGH 건수가 0 이상이면 빌드를 실패 처리하는 스크립트를 추가하세요.

```bash
# CI/CD 통합 예시 (Bash)
java -cp target/sast.jar com.sast.SASTEngine ./src ./report.md
CRITICAL=$(jq '[.findings[] | select(.severity=="CRITICAL")] | length' report.json)
if [ "$CRITICAL" -gt 0 ]; then
  echo "CRITICAL 취약점 ${CRITICAL}건 탐지 — 빌드 실패"
  exit 1
fi
```

Q. 규칙을 추가했는데 탐지가 안 됩니다.

A. `taintAnalysis: true`인 규칙은 `TaintAnalysisEngine.RULE_SINKS`에 해당 `ruleId`와 Sink 메서드 목록을 등록해야 합니다. 등록하지 않으면 Sink 탐지가 동작하지 않습니다. 재빌드(`mvn clean package`) 후 테스트하세요.

Java SAST Engine v1.0.0 — 행정안전부 소프트웨어 보안약점 진단가이드 2021 기반